VIT®
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

BCSE307L_COMPILER DESIGN

MODULE 3

Semantics Analysis

Dr. B.V. Baiju,
SCOPE,Assistant Professor
VIT, Vellore

Semantic Analysis

- Semantic Analysis makes sure that ***declarations and statements of program are semantically correct***.
- It ***uses syntax tree and symbol table*** to check whether the given program is semantically consistent with language definition.
- It gathers type information and stores it in either syntax tree or symbol table.

Semantic Errors:

- Errors recognized by semantic analyzer are as follows:
  - ***Type mismatch***
  - ***Undeclared variable***
  - ***Reserved identifier misuse.***
  - ***Multiple declaration of variable***
  - ***Accessing an out of scope varia***
  - ***Actual and formal parameter mismatch.***

Semantic Analysis

Syntax Directed Definition (SDD)

- A syntax-directed definition (SDD) is a ***context-free grammar together with attributes and rules***.

Attributes + CFG + Semantic Rules = Syntax Directed Definition

- **Attributes** are associated with grammar symbols (i.e. nodes of the parse tree)
  - The attribute of a grammar symbol can be ***numbers***, ***types***, ***table references***, or a ***string***.
- Rules are associated with productions.
- If ***X*** is a ***symbol*** and ***a*** is one of its ***attributes***, then we write ***X.a***

- This denotes the value a at a node in the parse-tree with the label X.

Production	Semantic Rules
E → E1 + T	E.val = E1.val+T.val
E → T	E.val = T.val

val is attribute

- Semantic rules setup dependencies between attributes which can represented by a dependency graph
- The dependency graph determines the evaluation order of these semantic rules
- Evaluation of a semantic rule defines the value of an attribute.

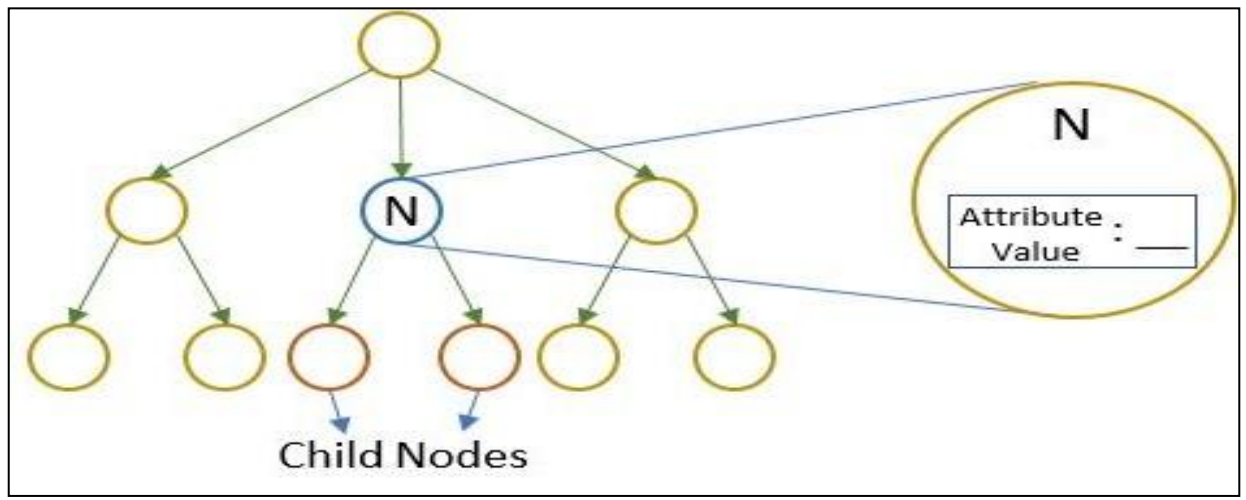
Inherited and Synthesised Attribute

- There are two kinds of attributes for the grammar symbol : for a non-terminal A, at a parse tree node N

a. Synthesized attribute (S- attribute)

- A non-terminal *A* at node *N* in a parse tree is defined by a semantic rule that is associated with the production at *N*.
- **Synthesized** attributes ***get values from the attribute values of their child nodes***.
- ***Bottom up manner*** (Child to parent node)

- Terminals can have synthesized attribute.
- Attributes can have lexical values and these values supplied by lexical analyzer
- ***Semantic rules can be applied only for non terminals for computing value of its attribute***



Example : **S → ABC**

S is said to be a synthesized attribute if it takes values from its child node (A, B, C).

- A syntax directed definition that uses synthesized attribute exclusively is said to be **S-attribute definition**.

n : Used to terminate the evaluation

Lexval : Lexical value, value for this terminal is passed by lexical analyzer

L → En : Augmented Grammar

val : Attribute

E.val = T.val : E.val used to define L.val

E.val = E_1.val + T.val : E. val can be derived from adding E.val with T.val

Production	Semantic rules
L → En	Print (E.val)
E → E_1+T	E.val = E_1.val + T.val
E → T	E.val = T.val
T → T_1*F	T.val = T_1.val * F.val
T → F	T.val = F.val
F → (E)	F.val = E.val
F → digit	F.val = digit.lexval

Syntax directed definition of simple desk calculator which evaluates expressions terminated by an endmarker, n

Evaluating an SDD at the Nodes of a Parse Tree

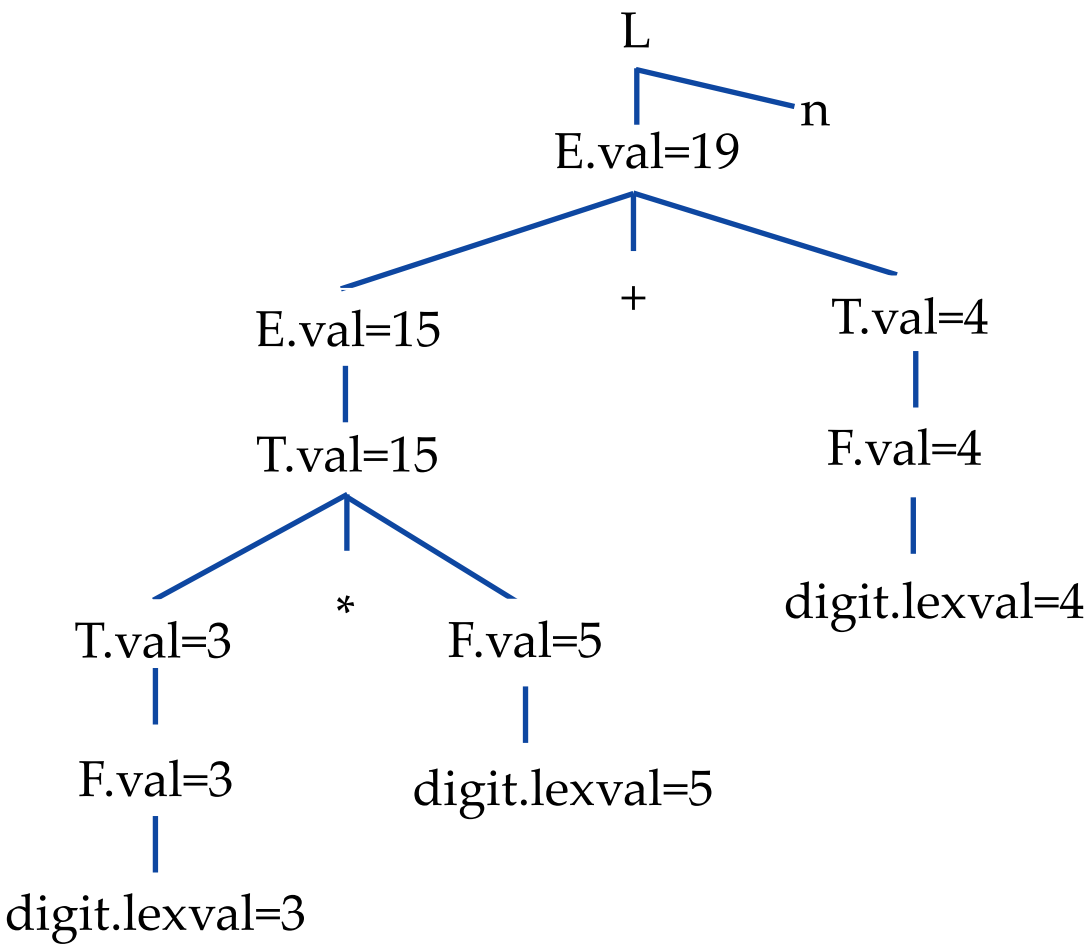
- To determine the value of attributes at nodes in a parse tree, do the following:
  - We must first construct a parse tree.
  - Then we must use SDD rules to assess the values of all the nodes in the parse tree's attributes.
  - An annotated parse tree is formed via this improvised parse tree.

Construct Annotated Parse Tree

- Annotated Parse tree contains the values and attributes at each node.
- Before analyzing a node's attribute value, first evaluate all the elements that determine its value.
  - To evaluate a ***node's synthesized attribute***, parse the tree from the **bottom up**.
    - Its value is determined by the value of the child attributes of the concerned node and the node itself.
  - To determine a ***node's inherited attribute***, parse the tree from the **top down**.
    - Its value is determined by its parent, siblings, and the node itself.
  - Synthesized and inherited attributes may coexist at some nodes in a parse tree. In this scenario, we're not sure if there is even a single order in which the nodes' characteristics may be evaluated.

Example: Synthesized attributes

String: 3*5+4n;

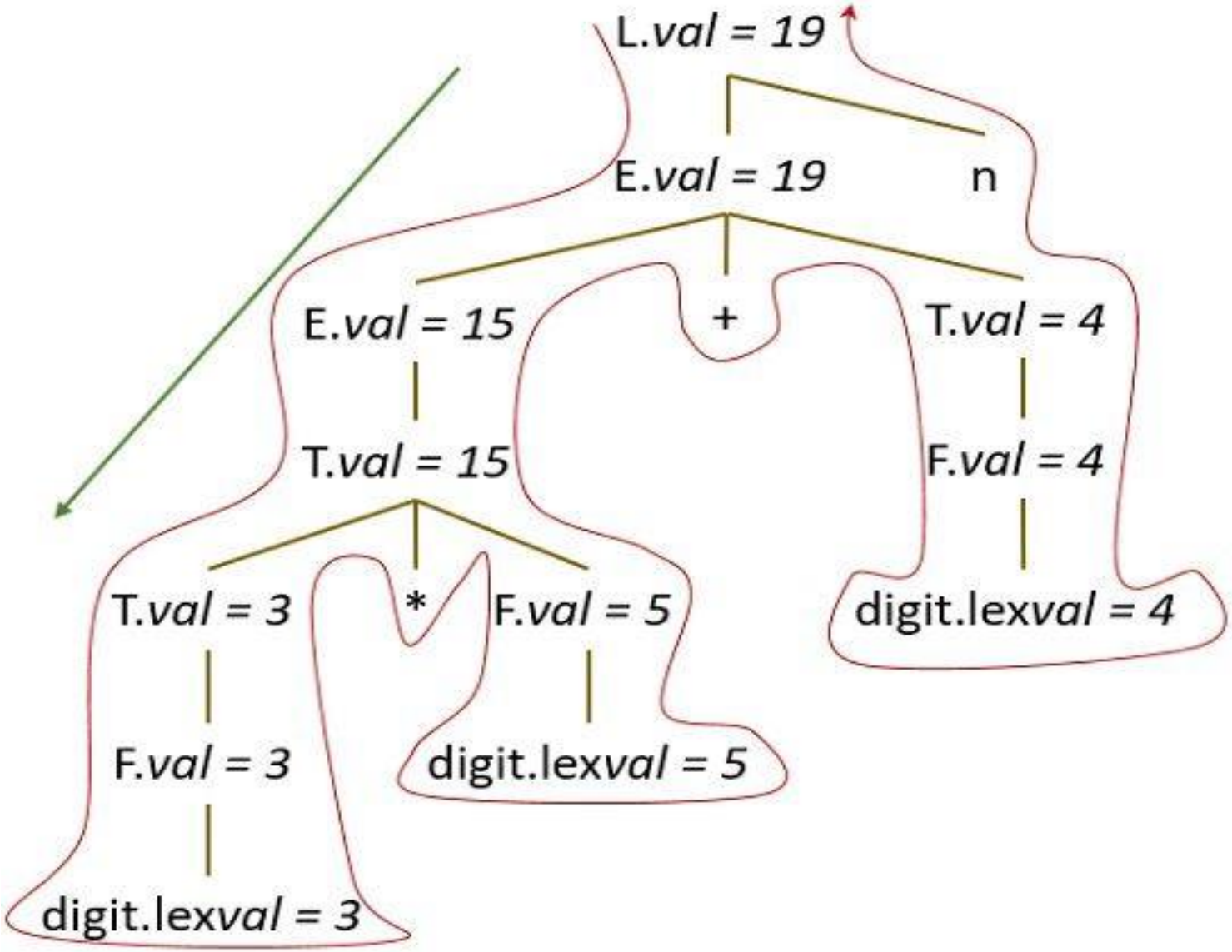


Annotated parse tree for 3*5+4n

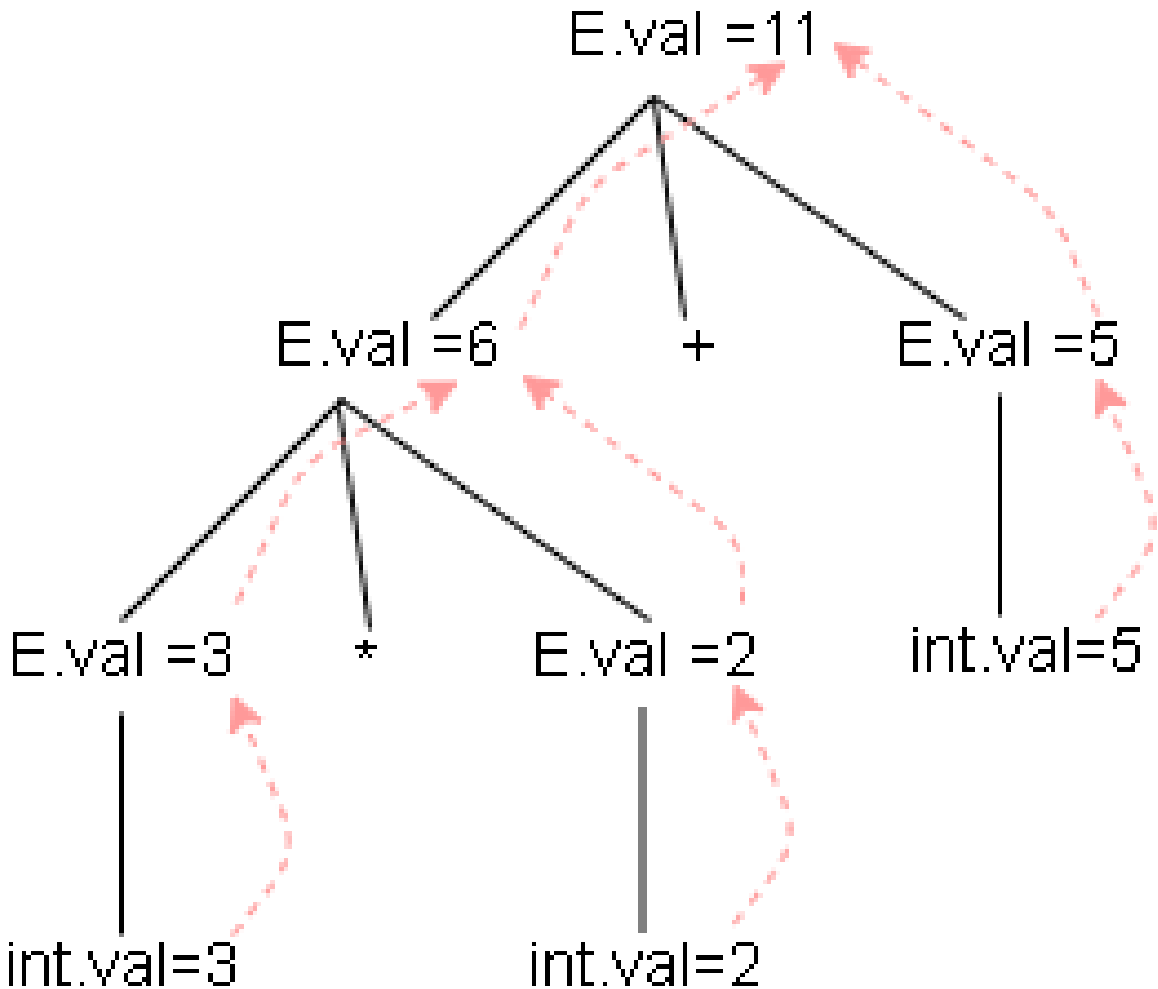
Production	Semantic rules
L → E_n	Print (E.val)
E → E_1+T	E.Val = E_1.val + T.val
E → T	E.Val = T.val
T → T_1*F	T.Val = T_1.val * F.val
T → F	T.Val = F.val
F → (E)	F.Val = E.val
F → digit	F.Val = digit . lexval

The process of computing the attribute values at the node is called annotating or decorating the parse tree

parse tree showing the value of the attributes at each node is called Annotated parse tree



Annotated Parse Tree for Input String 3 * 5 + 4 n


E.val =11
E.val =6
+
E.val =5
E.val =3
*
E.val =2
int.val=5
int.val=3
int.val=2

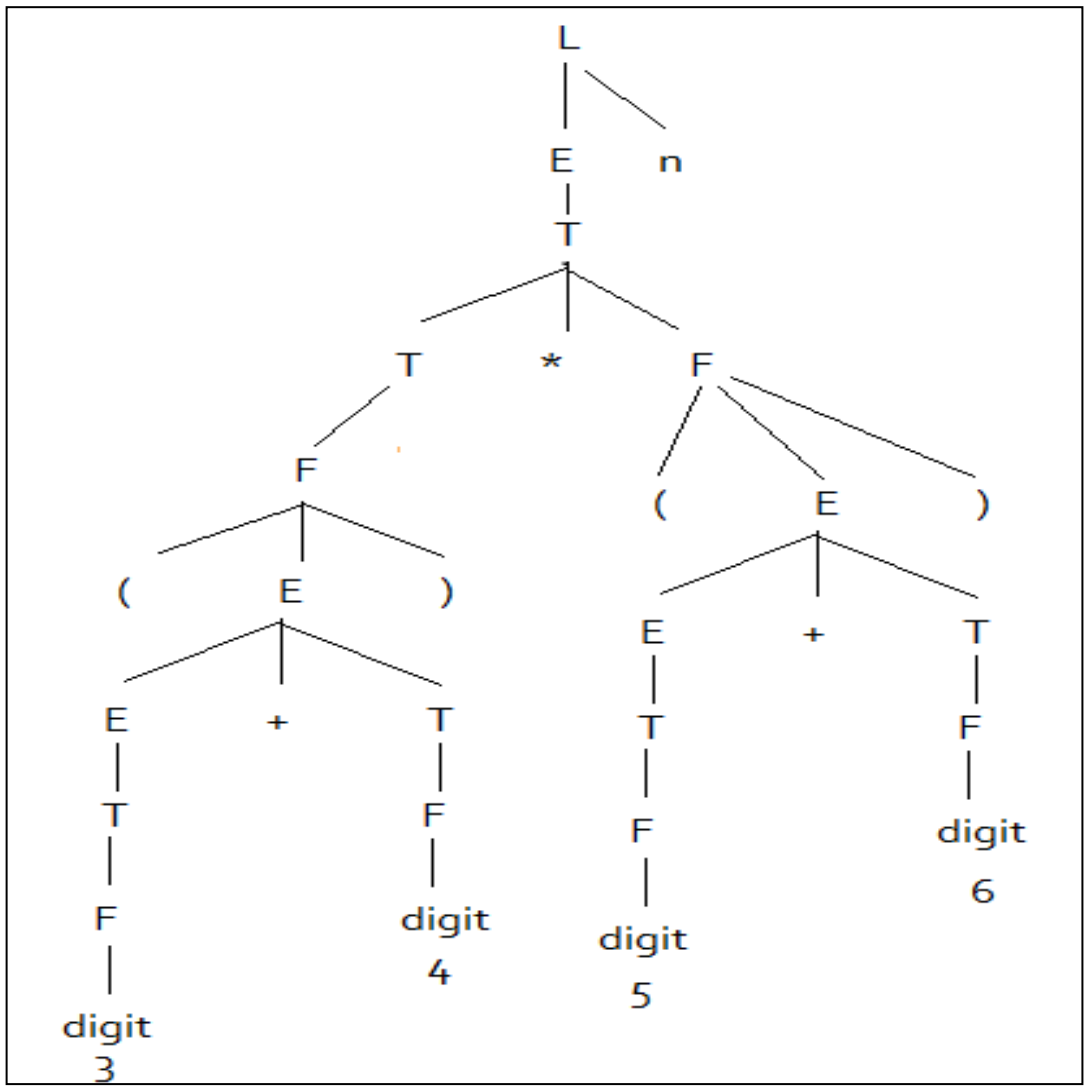
Exercise

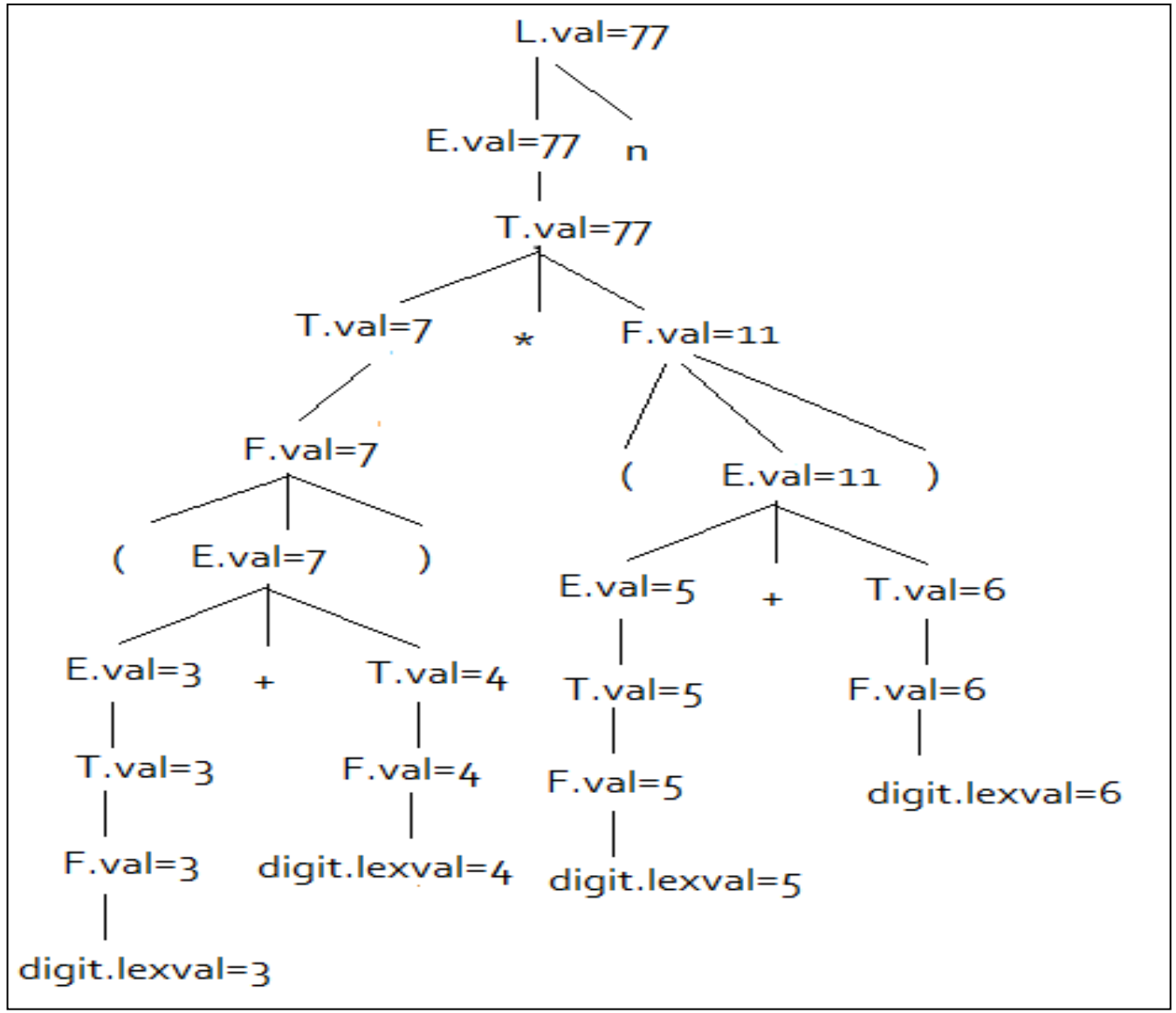
Draw Annotated Parse tree for following using the SDD:

1. **(3+4)*(5+6)n**
2. **1*2*3*(4+5)n**
3. **(9+8*(7+6)+5)*4n**

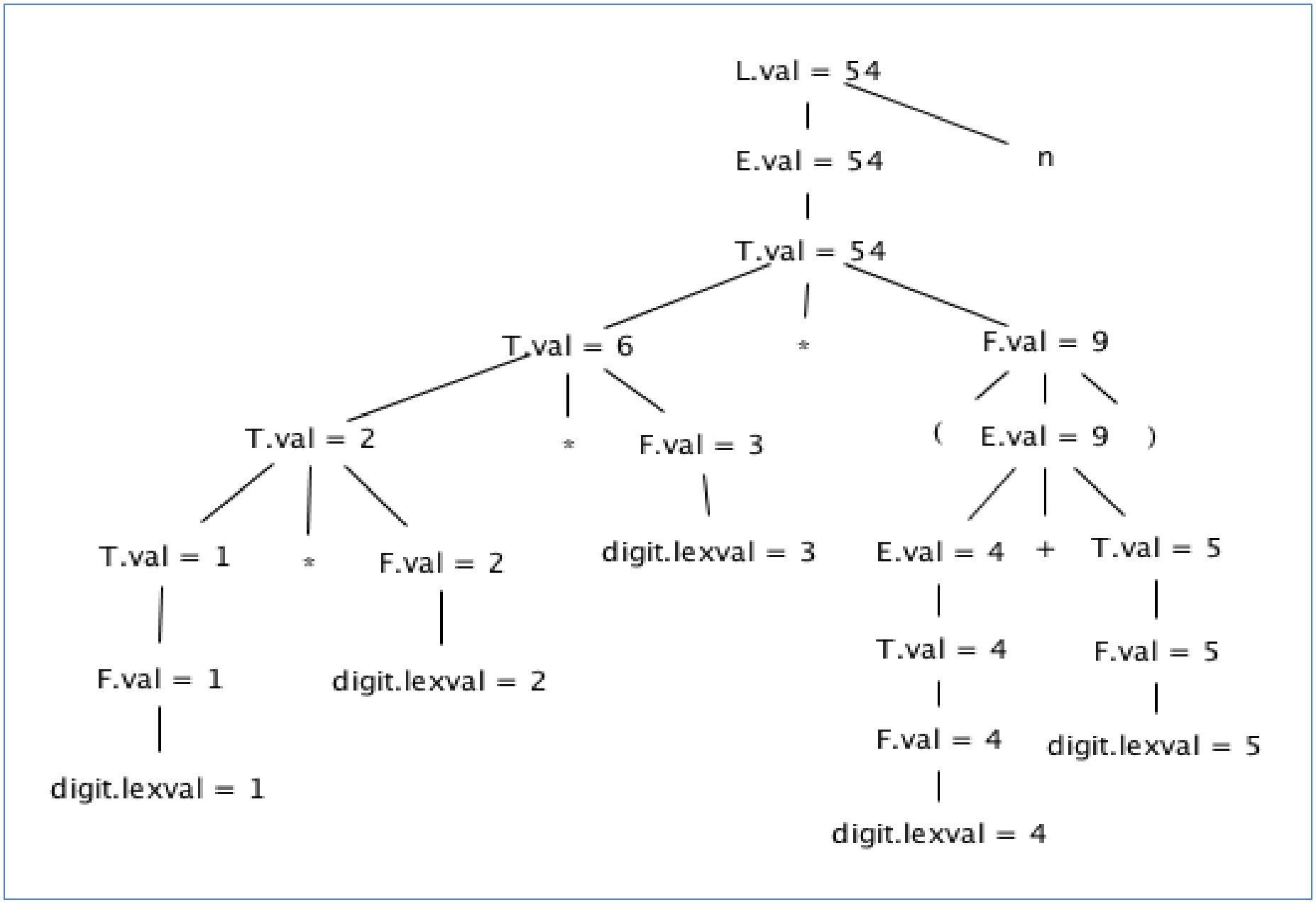
Production	**Semantic rules**
L → E_n	Print (E.val)
E → E_1+T	E.Val = E_1.val + T.val
E → T	E.Val = T.val
T → T_1*F	T.Val = T_1.val * F.val
T → F	T.Val = F.val
F → (E)	F.Val = E.val
F → digit	F.Val = digit . lexval

(3+4)*(5+6)n


L
E
n
T
T
*
F
F
(
E
)
(
E
)
E
+
T
E
+
T
T
F
T
F
F
digit
6
F
digit
4
digit
5
F
digit
3

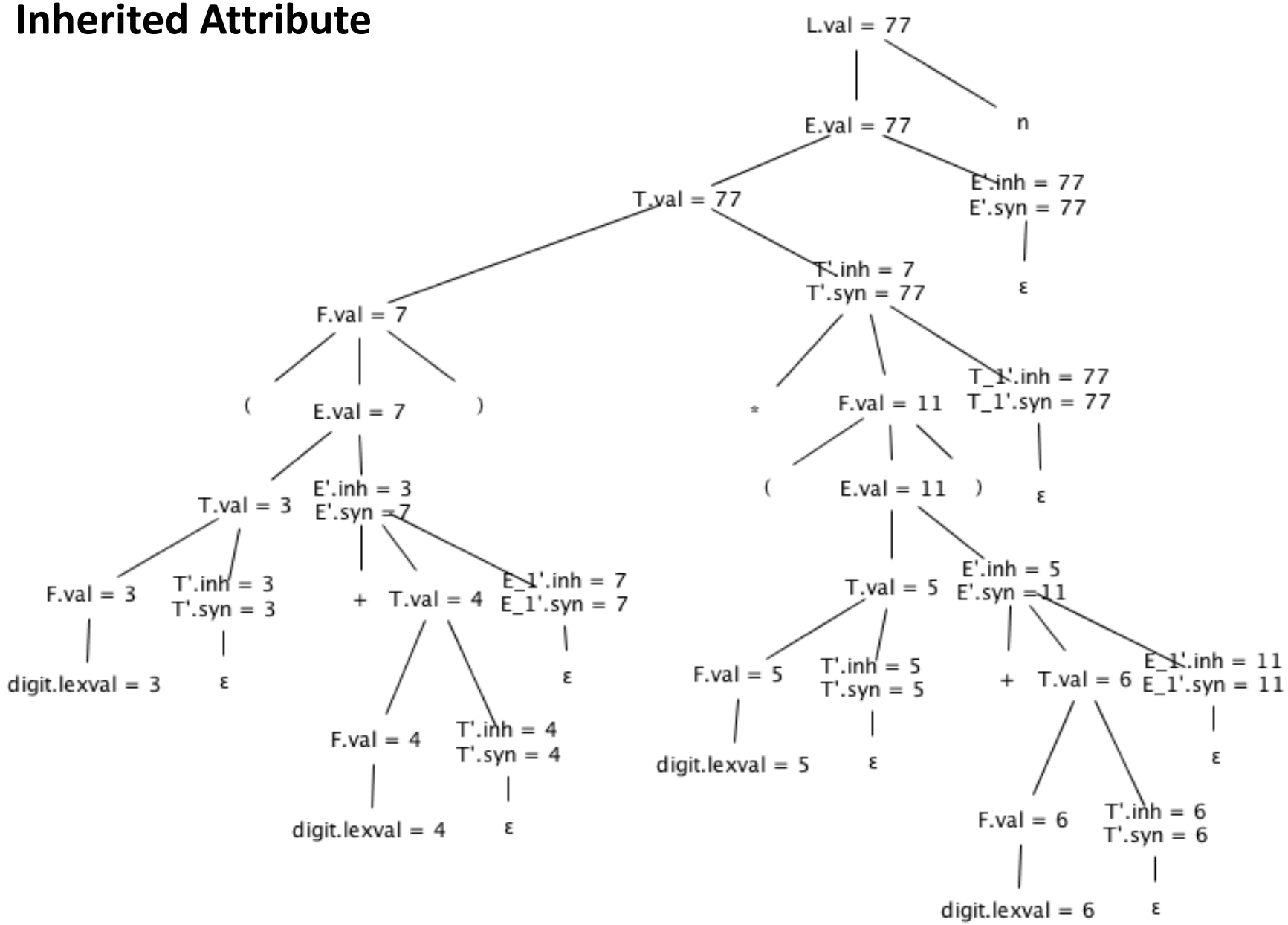

L.val=77
E.val=77
n
T.val=77
T.val=7
*
F.val=11
F.val=7
(
E.val=11
)
(
E.val=7
)
E.val=5
+
T.val=6
E.val=3
+
T.val=4
T.val=5
F.val=6
T.val=3
F.val=4
F.val=5
digit.lexval=6
F.val=3
digit.lexval=4
digit.lexval=5
digit.lexval=3

1*2*3*(4+5)n

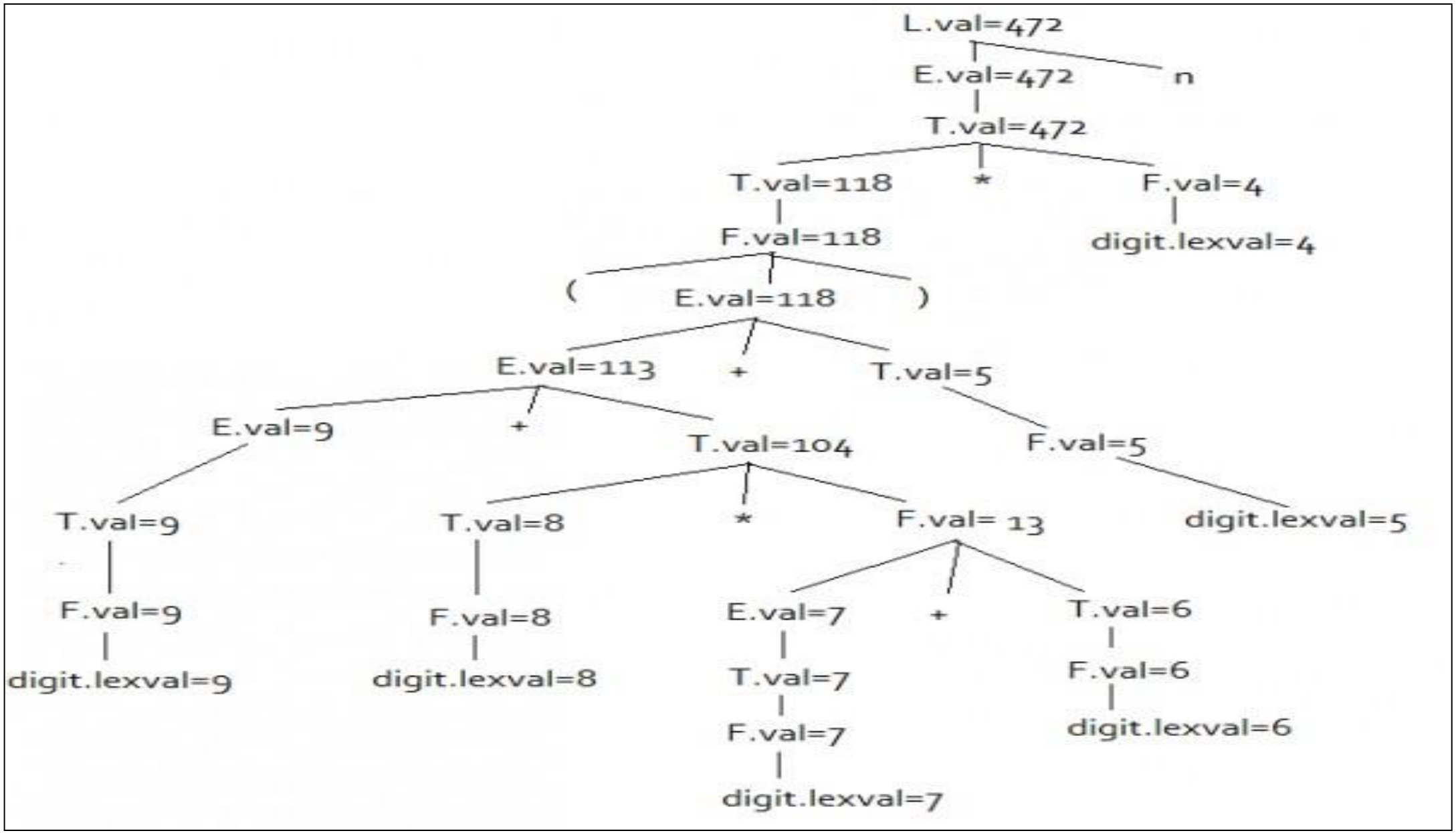

L.val = 54
E.val = 54
n
T.val = 54
T.val = 6
*
F.val = 9
T.val = 2
*
F.val = 3
(
E.val = 9
)
T.val = 1
*
F.val = 2
digit.lexval = 3
E.val = 4
+
T.val = 5
F.val = 1
digit.lexval = 2
T.val = 4
F.val = 5
digit.lexval = 1
F.val = 4
digit.lexval = 5
digit.lexval = 4

(3+4)*(5+6)n

Inherited Attribute

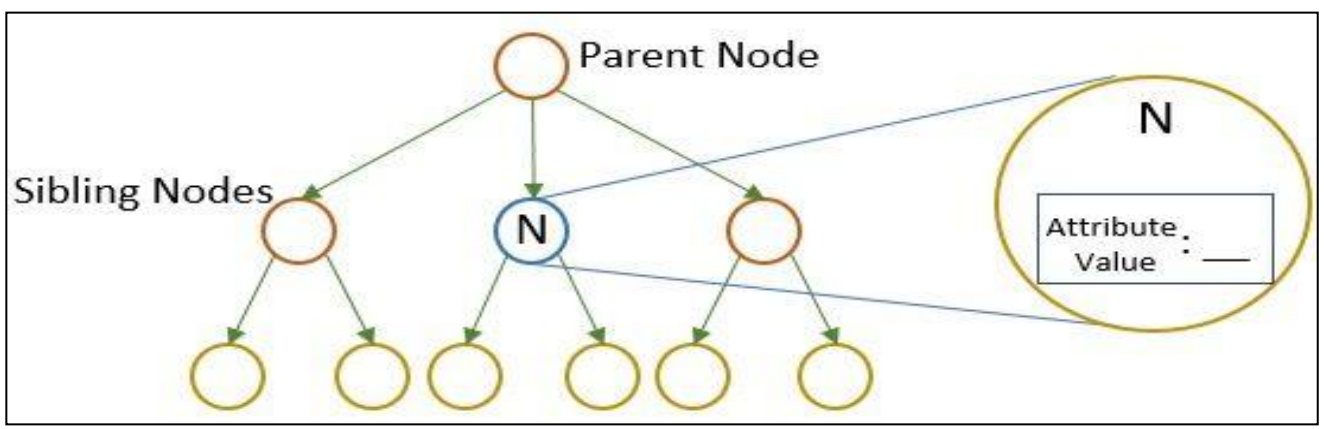


(9+8*(7+6)+5)*4n


L.val=472
E.val=472
n
T.val=472
T.val=118
*
F.val=4
F.val=118
digit.lexval=4
(
E.val=118
)
E.val=113
+
T.val=5
E.val=9
+
T.val=104
F.val=5
T.val=9
T.val=8
*
F.val= 13
digit.lexval=5
F.val=9
F.val=8
E.val=7
+
T.val=6
digit.lexval=9
digit.lexval=8
T.val=7
F.val=6
F.val=7
digit.lexval=6
digit.lexval=7

b. Inherited attribute (I-attribute)

- For a nonterminal B at a parse-tree node N is defined by a semantic rule associated with the production at the parent of N.
- The value of inherited attribute is computed from the values of attributes at the siblings and parent of that node.
- The value of an inherited attribute for a non-terminal symbol (node) N can be defined as
  - **The attribute values of N's *parent*.**
  - **Attribute values of N's *Siblings*.**
  - **Total attribute values of N *itself*.**
- ***Top down manner*** (Parent to child)



Example;

S → ABC

A can get its values from S, B and C
B can get its values from S, A and C
C can get its values from A, B and S

Syntax Directed Definitions are used to specify syntax directed translation.

- There are 2 types of SDD

S – Attributed Definitions

L - Attributed Definitions

S – Attributed Definitions

- Only ***synthesized attributes*** used in syntax directed definition.
- S-attributed grammars interact well with ***LR(K) parsers***, since the evaluation of attributes is bottom-up.

L - Attributed Definitions

- Uses both ***synthesized attribute*** and ***inherited attribute***
- An inherited value at a node in a parse tree is computed from the value of attributes at the parent and/or only left siblings of the node.
- L-attributed grammars interact well with ***LL(K) parsers***

L-Attributed Definitions

Definition: **An SDD is *L-Attributed* if each attribute is either**

1.Synthesized.

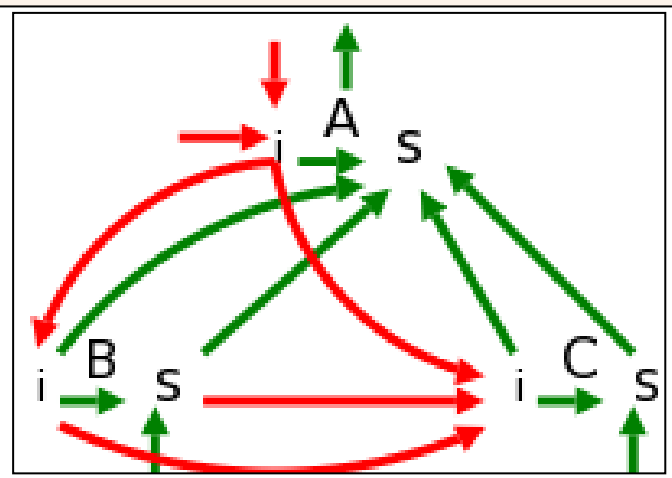
2.Inherited from the left, and hence the name L-attributed.

If the production is $A \rightarrow X_1X_2...X_n$, then the inherited attributes for X_j can depend only on

a.Inherited attributes of A, the LHS.

b.Any attribute of X_1, …, X_{j-1}, i.e. only on symbols to the left of X_j.

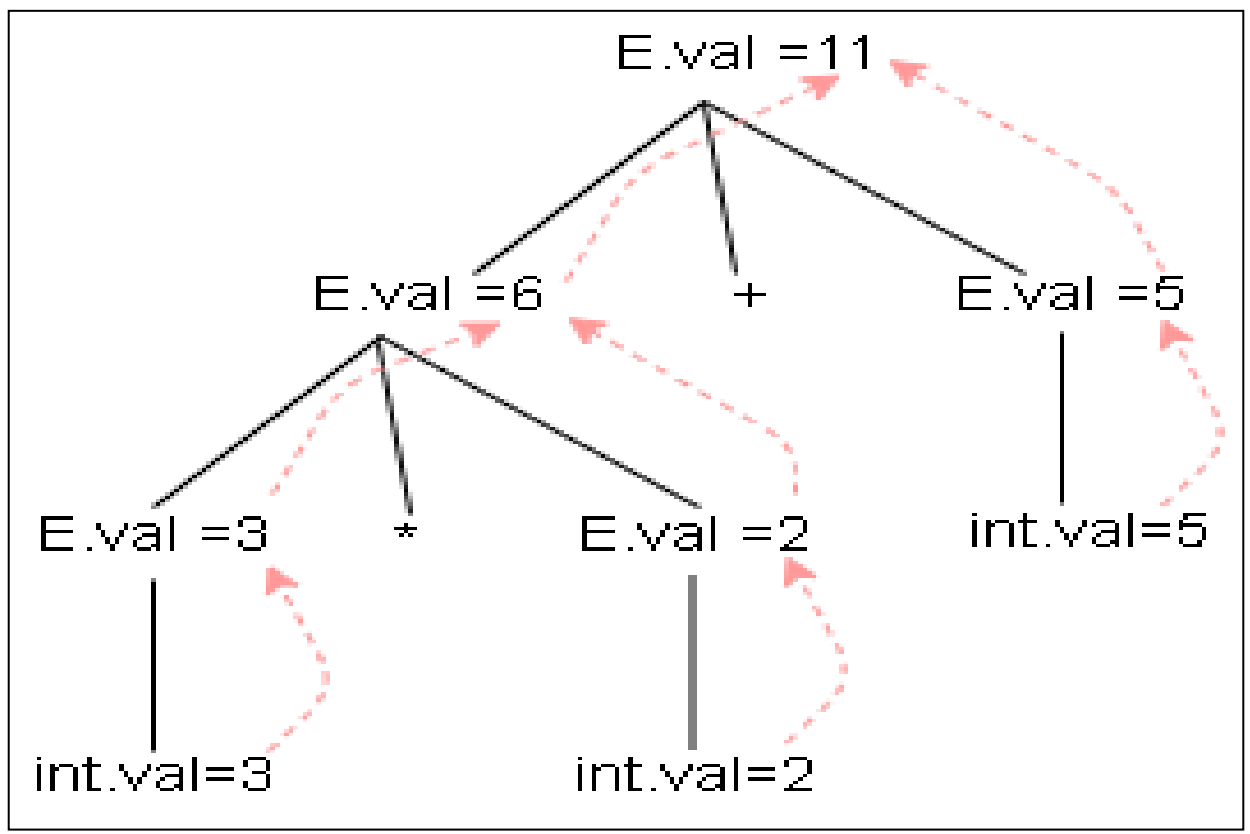
3.Attributes of X_j, ***BUT*** you must guarantee (separately) that these attributes do not by themselves cause a cycle.



Example for synthesized attribute:

Production	Semantic Rules
E → E1 * E2	{E.val = E1.val * E2.val}
E → E1 + E2	{E.val = E1.val +E2.val}
E → int	{E.val = int.val}

String: 3*2+5



An SDD based on a grammar suitable for top-down parsing

T → T * F
T → F
F → digit

	PRODUCTION	SEMANTIC RULES
1)	$T \rightarrow F\ T'$	$T'.inh = F.val$ $T.val = T'.syn$
2)	$T' \rightarrow * F\ T'_1$	$T'_1.inh = T'.inh \times F.val$ $T'.syn = T'_1.syn$
3)	$T' \rightarrow \epsilon$	$T'.syn = T'.inh$
4)	$F \rightarrow \mathbf{digit}$	$F.val = \mathbf{digit}.lexval$

- Grammar contains 3 non terminal **T**, **F**, **T'**
- ***digit*** has attribute value (i.e lexval)

- ***val*** : Synthesized attribute of non terminal T, F (T. val and F.val)
- ***T'*** has 2 attribute
  - **inh** inherited attribute
  - **syn** synthesized attribute

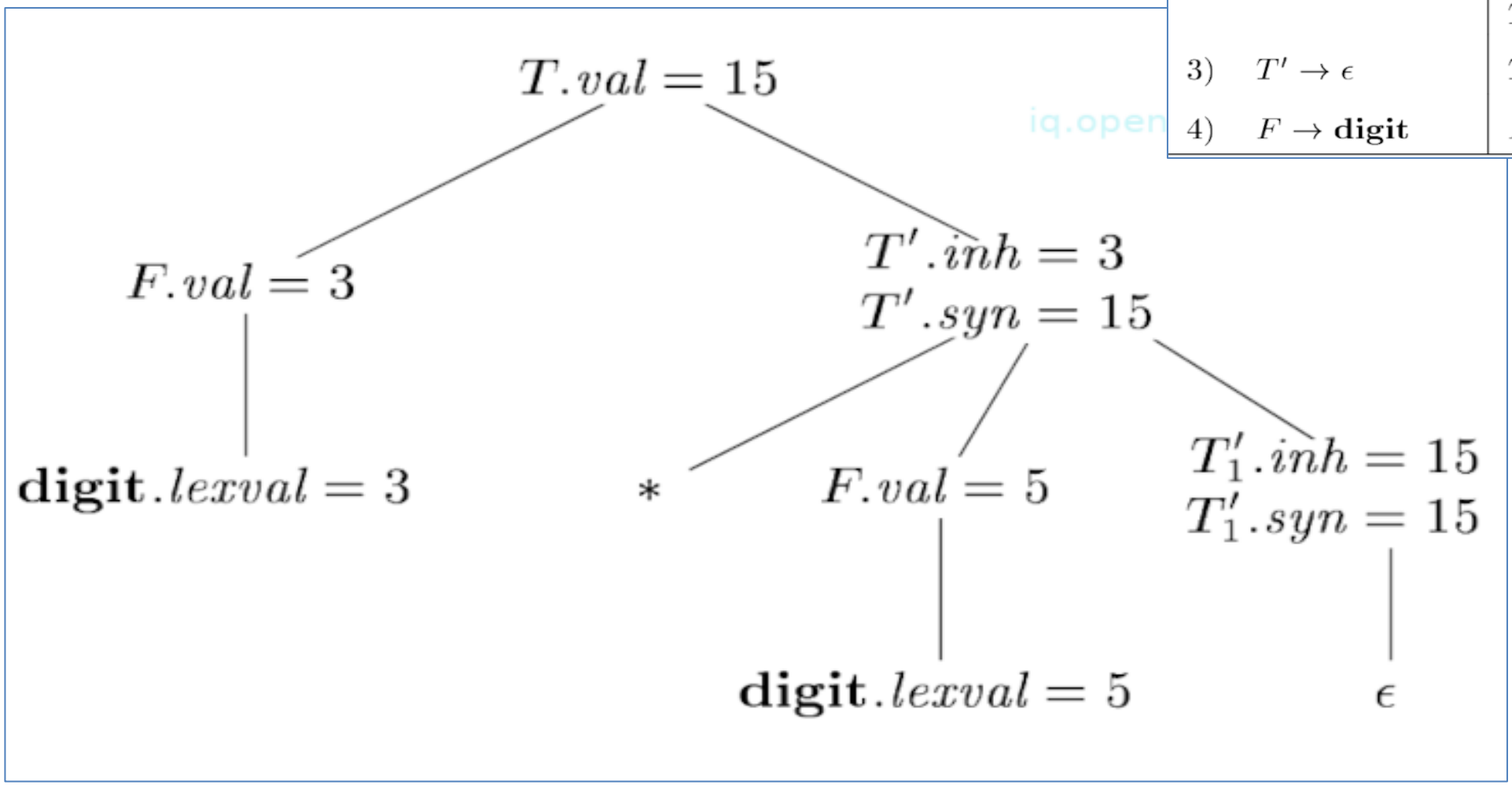
T → FT' (T' can be inherited from F, T can have the value from T')
T' → *FT' (T' can be inherited from F)

	Semantic Rule	Production
F.val = 2 is copied to T′.inh	T′.inh = F.val	T → F T′
T′.inh * F.val is copied to T_1′.inh	T_1′.inh = T′.inh * F.val	T′ → * F T′
T_1′.inh is copied to T′.syn	T′.syn = T_1′.inh	T′ → ϵ
T′.syn is moved to its parent	T′.syn = T_1′.syn	T′ → * F T′
T′.syn is moved to its parent T	T.val = T′.syn	T → F T′

Production	Semantic Rule	Type
$S \rightarrow E\ n$	$S.v = E.v$	Synthesized
$E \rightarrow T\ E'$	$E'.inh = T.val$	Inherited
	$E.val = E'.syn$	Synthesized
$E' \rightarrow +\ T\ E'$	$E_1'.inh = E'.inh + T.val$	Inherited
	$E'.syn = E_1'.syn$	Synthesized
$E' \rightarrow \epsilon$	$E'.syn = E_1'.inh$	Synthesized
$T \rightarrow F\ T'$	$T'.inh = F.val$	Inherited
	$T.val = T'.syn$	Synthesized
$T' \rightarrow *\ F\ T'$	$T_1'.inh = T'.inh * F.val$	Inherited
	$T'.syn = T_1'.syn$	Synthesized
$T' \rightarrow \epsilon$	$T'.syn = T_1'.inh$	Synthesized
$F \rightarrow (\ E\ )$	$F.v = E.v$	Synthesized
$F \rightarrow d$	$F.v = digit.lexval$	Synthesized

Construct Annotated parse tree for 3 * 5

	Production	Semantic Rules
1)	$T \rightarrow F\ T'$	$T'.inh = F.val$ $T.val = T'.syn$
2)	$T' \rightarrow *\ F\ T'_1$	$T'_1.inh = T'.inh \times F.val$ $T'.syn = T'_1.syn$
3)	$T' \rightarrow \epsilon$	$T'.syn = T'.inh$
4)	$F \rightarrow \mathbf{digit}$	$F.val = \mathbf{digit}.lexval$



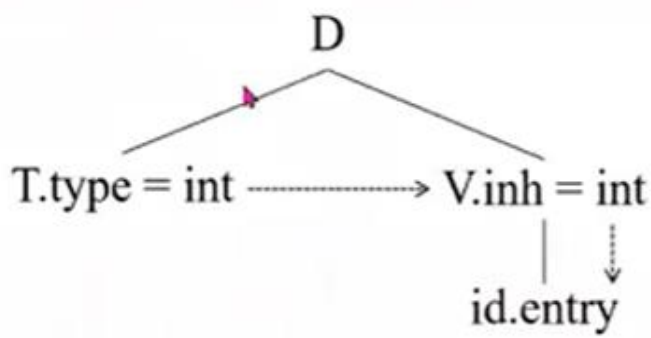
- An inherited attribute distributes type information to the various identifiers in a declaration.
- Consider the production **D → T V** which is used for a single declaration such as

int sum

D stands for **declaration**
T stands for **type** (int)
V stands for **variable** (Sum)

Production	Semantic Rules
D → T V	V.inh = T.type

Parse tree with attribute values:

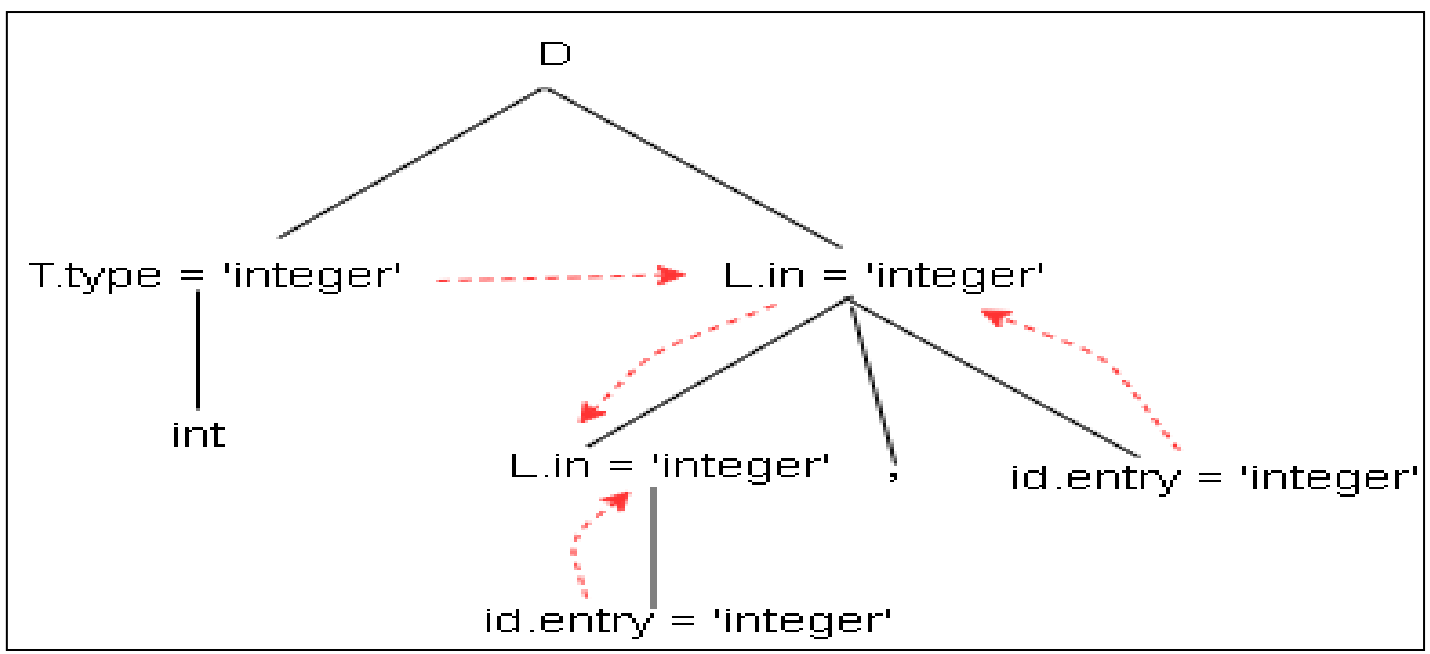


Example for Inherited Attributes:

Production	Semantic Rules
D → TL	L.in = T.type
T → int	T.type = integer
T → real	T.type = real
L → L1, id	L1.in = L.in; addtype(id.entry, L.in)
L → id	addtype(id.entry, L.in)

- The keyword **int** or **real** followed by a list of identifiers.
- **Symbol T** is associated with a **synthesized attribute type**
- **Symbol L** is associated with an **inherited attribute L.in**.
- Rules associated with L call for procedure add type to the type of each identifier to its entry in symbol table

Input: int id, id

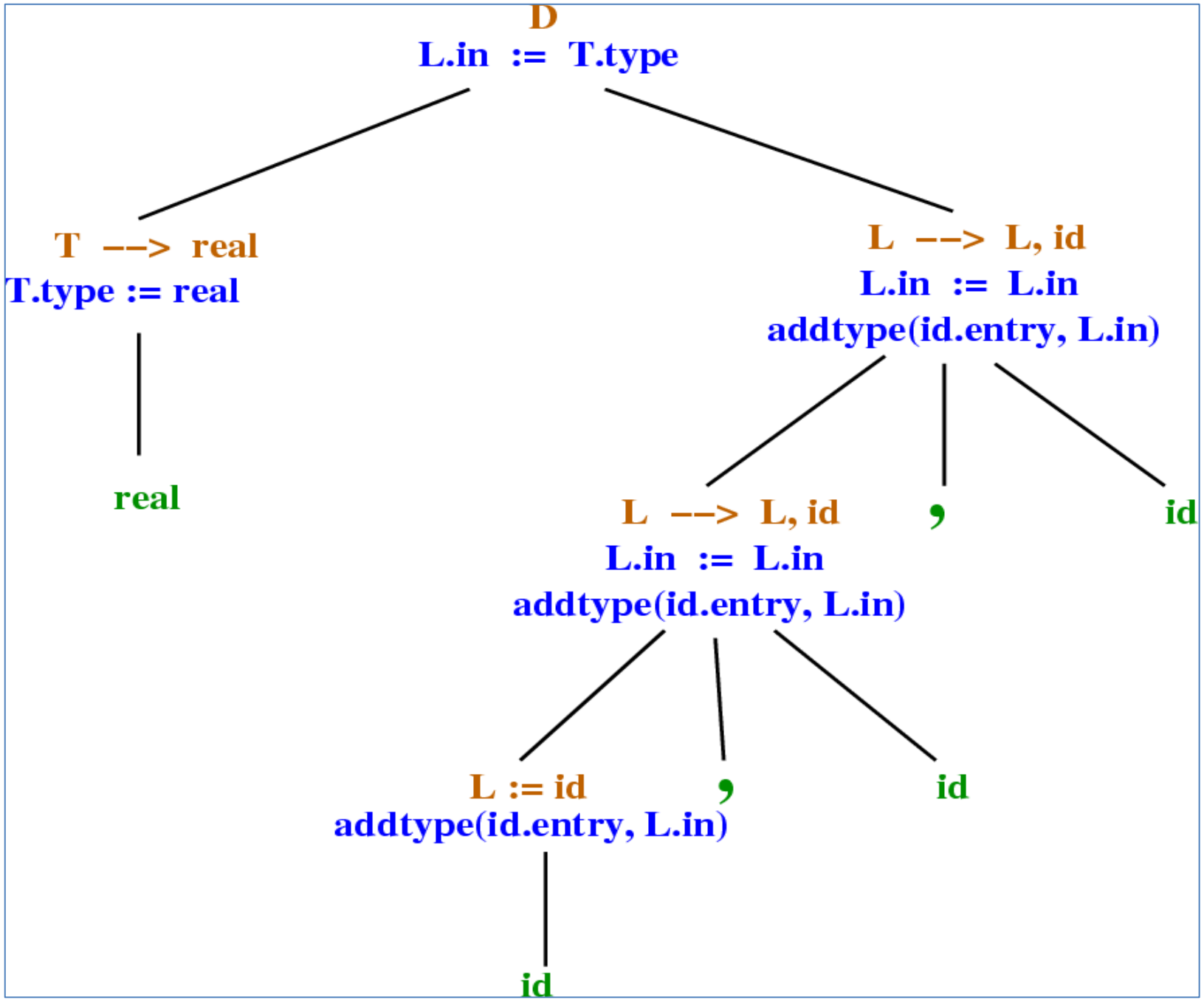


- The nonterminal T has a synthesized attribute whose value is received by the inherited attribute of L via the semantic rule

  L.in := T.type.

- Then the type L.in is passed down a parse tree via the semantic rule

  L1.in := L.in.



Evaluation Orders for SDD's

- Dependency graphs are a useful tool for ***determining an evaluation order for the attribute instances in a given parse tree*.**
- Semantic rules set up dependencies between attributes which can be represented by a dependency graph.
- While an **annotated parse tree** shows the ***values of attributes***, a **dependency graph** helps us ***determine how those values can be computed***.
- It is a **Directed Graph**
- ***If an attribute b at a node depends on an attribute c, then the semantic rule for b at that node must be evaluated after the semantic rule that defines c.***
- Construction:
  - Put each semantic rule into the form **$b=f(c_1,\ldots,c_k)$** by introducing dummy synthesized attribute b for every semantic rule that consists of a procedure call.
  - E.g.,

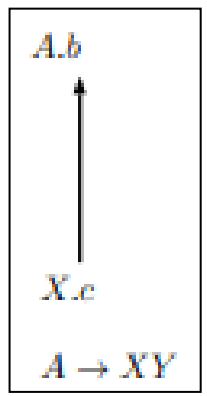
L → E n	**print(E.val)**
Becomes:	**dummy = print(E.val)**

  - The graph has a node for each attribute and an edge to the node for b from the node for c if attribute b depends on attribute c.

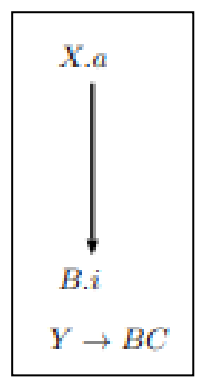
- A dependency graph indicate the flow of information among the instances of attributes in a given parse tree.
- An edge from one attribute instance to another means that the value of the first is needed to compute the second.
- Edges express constraints implied by the semantic rules

For each parse-tree node, say a node labeled by grammar symbol X, the dependency graph has a node for each attribute associated with X.

Suppose that a semantic rule associated with a production p defines the value of synthesized attribute A.b in terms of the value of X.c (the rule may define A.b in terms of other attributes in addition to X.c) . Then, the dependency graph has an edge from X.c to A.b. More precisely, at every node N labeled as A where production p is applied, create an edge to attribute b at N, from the attribute c at the child of N corresponding to this instance of the symbol X in the body of the production1 .

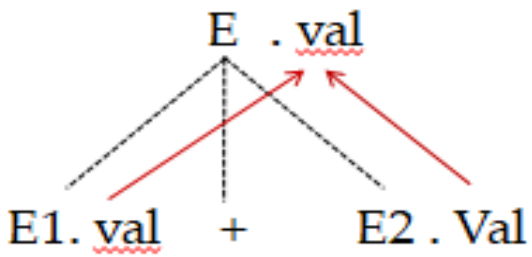


Suppose that a semantic rule associated with a production p defines the value of inherited attribute B.i in terms of the value of X.a. Then, the dependency graph has an edge from X.a to B.i. For each node N labeled B that corresponds to an occurrence of this B in the body of production p, create an edge to attribute i at N from the attribute a at the node M that corresponds to this occurrence of X. Note that M could be either the parent or a sibling of N.



- Example

Production	Semantic Rule
E→E1 + E2	**E.val = E1.val + E2.val**



- **E.val** is synthesized from E1.val and E2.val
- The dotted lines represent the parse tree that is not part of the dependency graph.

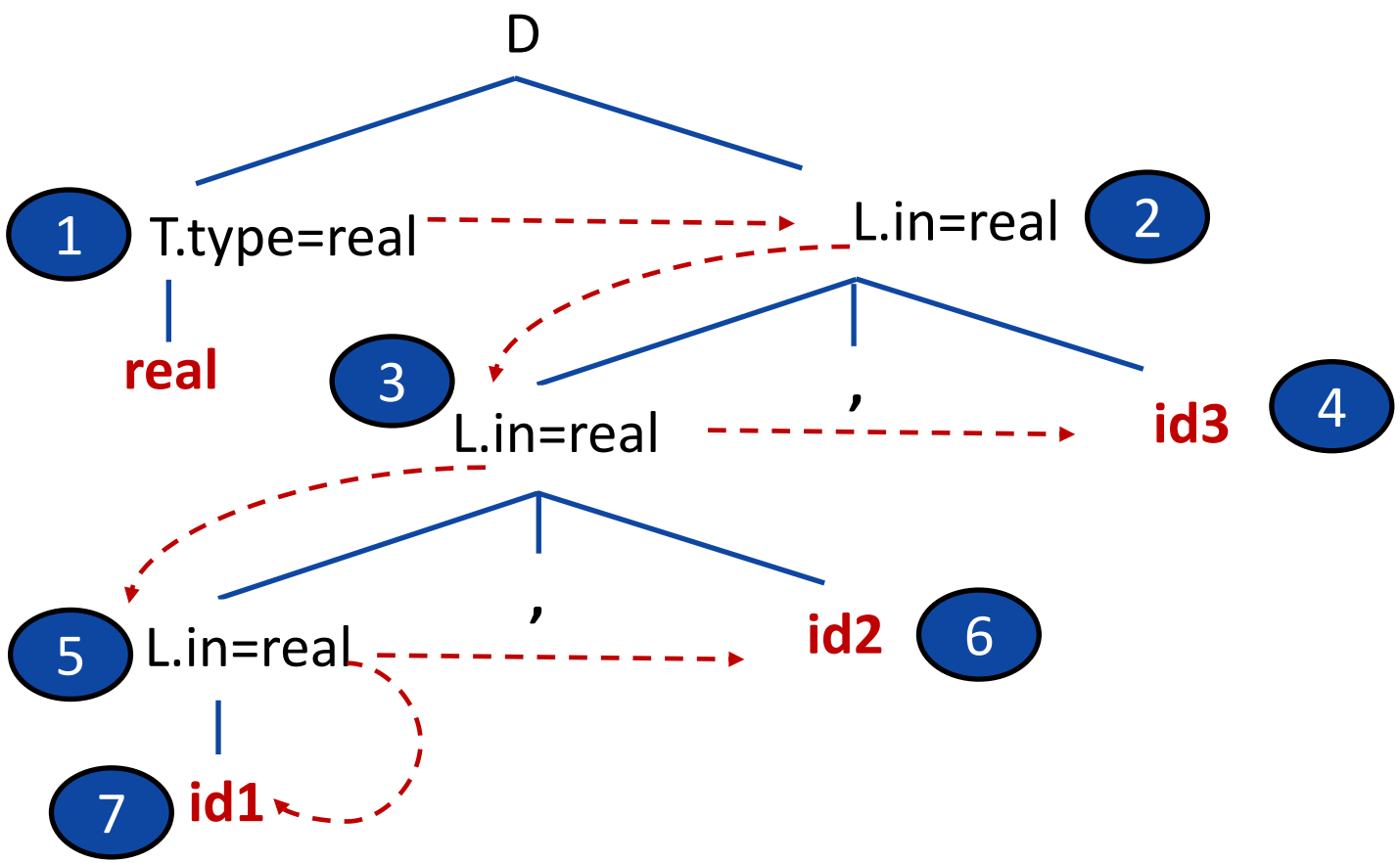
Dependency Graph Construction

for each node *n* in the parse tree do
    for each attribute a of the grammar symbol at *n* do
        construct a node in the dependency graph for a

for each node *n* in the parse tree do
    for each semantic rule $b = f(c_1,\ldots,c_n)$
        associated with the production used at *n* do
            for i= 1 to n do
                construct an edge from the node c_i to the node *b*

Evaluation order

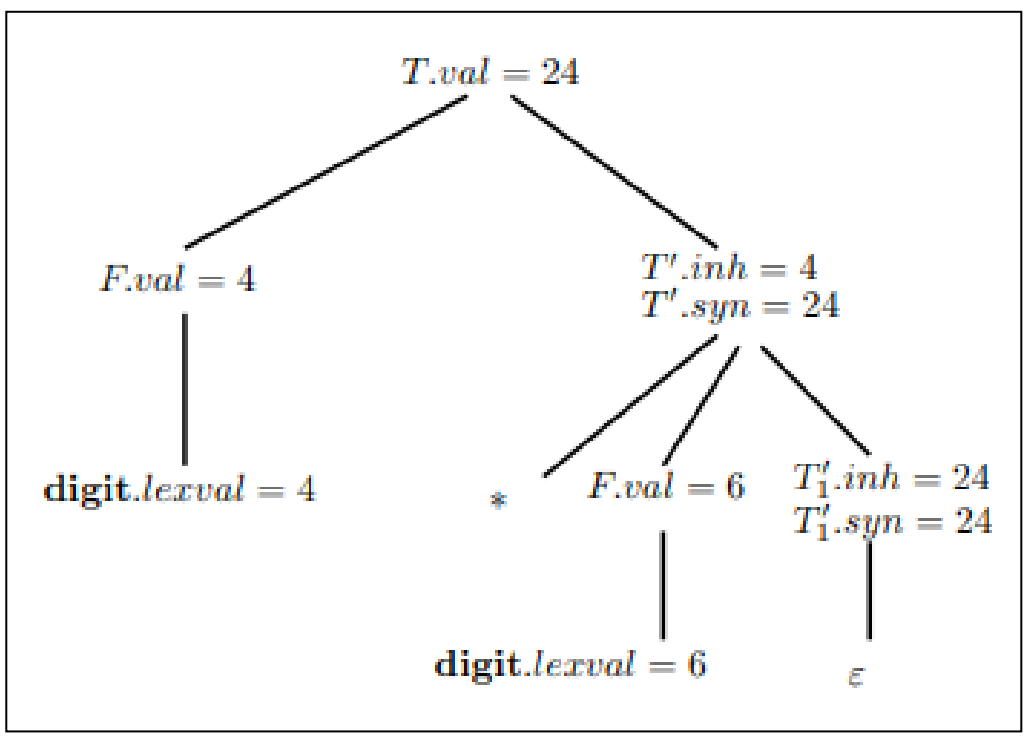
- A topological sort of a directed acyclic graph is any ordering $m_1, m_2, \ldots\ldots\ldots, m_k$ of the nodes of the graph such that edges go from nodes earlier in the ordering to later nodes.
- If $m_i \rightarrow m_j$ is an edge from m_i to m_j then m_i appears before m_j in the ordering.

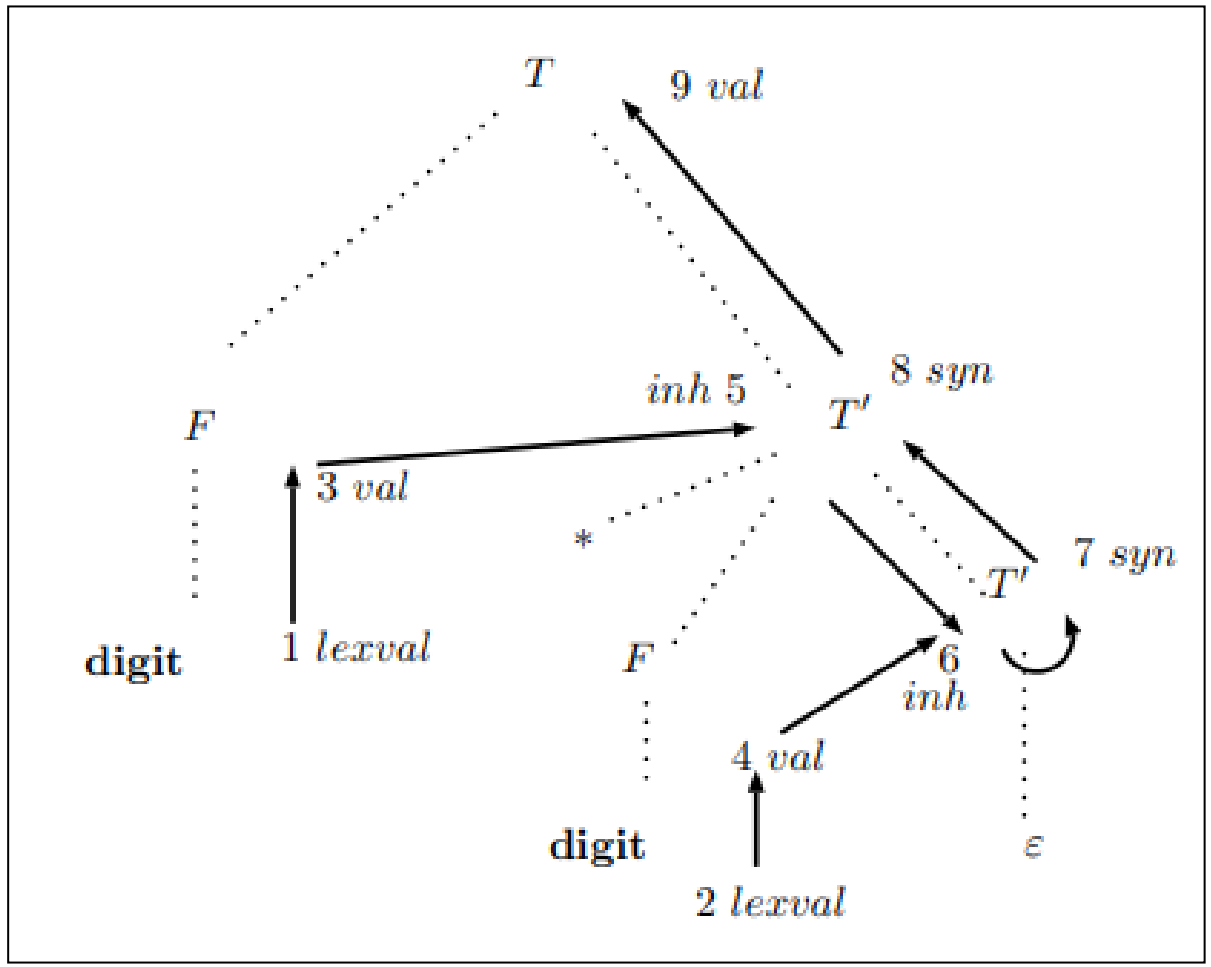


Example:

Construct a Dependency graph for the annotated parse tree, whose SDD is given in Table . The input string is 4 * 6

S.No.	Production	Semantic Rules
1.	$T \rightarrow FT'$	$T'.inh = F.val$ $T.val = T'.syn$
2.	$T' \rightarrow *FT'_1$	$T'_1.inh = T'_1.inh \times F.val$ $T'.syn = T'_1.syn$
3.	$T' \rightarrow \varepsilon$	$T'.syn = T'.inh$
4.	$F \rightarrow$ **digit**	$F.val =$ **digit**$.lexval$

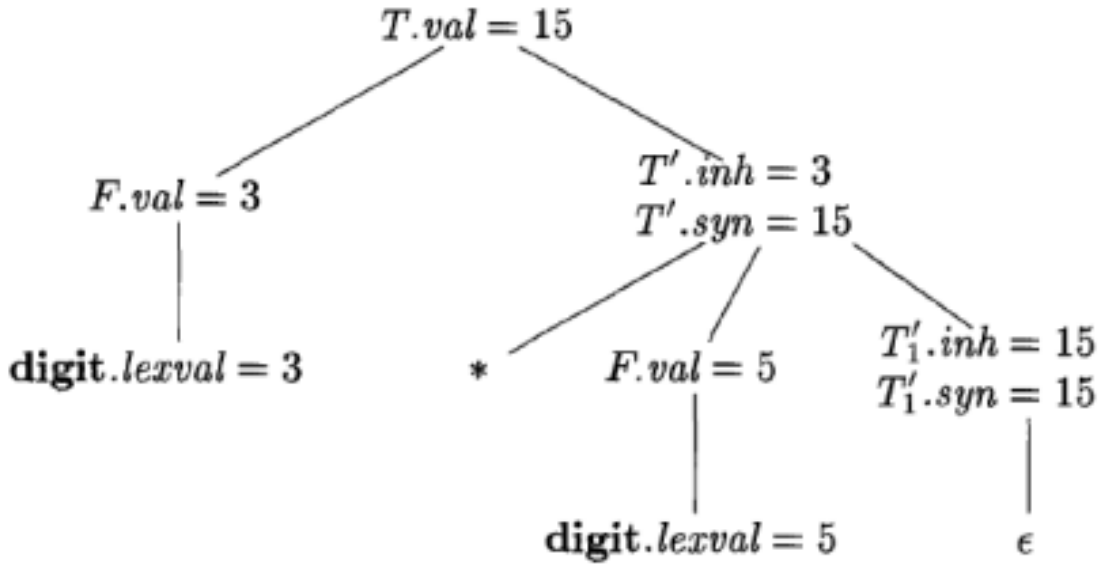


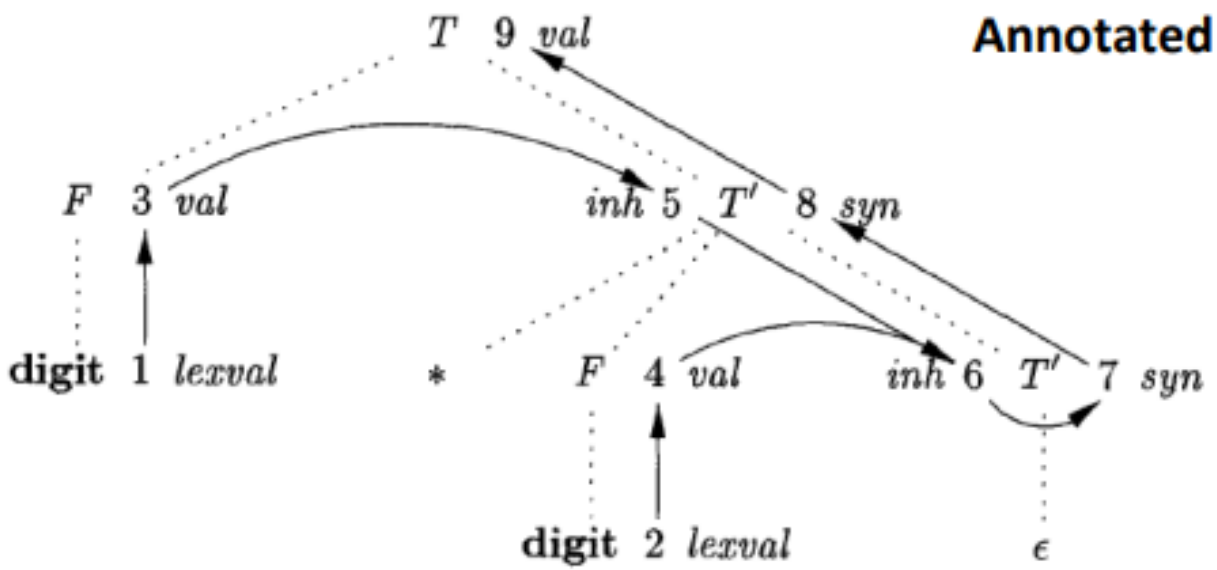


The nodes of the dependency graph, represented by the numbers 1 through 9

- Nodes 1 and 2 represent the attribute lexval associated with the two leaves labeled digit.
- Nodes 3 and 4 represent the attribute val (i.e., 4 and 6) associated with the two nodes labeled F.
- The edges to node 3 from 1 and to node 4 from 2 result from the semantic rule that defines F.val in terms of digit.lexua1.
- In fact, F.val equals digit.lexval, but the edge represents dependence, not equality.
- Nodes 5 and 6 represent the inherited attribute T ′ .inh associated with each of the occurrences of nonterminal T ′ (T ′ → ∗F T ′ 1 and T ′ → ε).
- The edge from 3 to 5 is due to the rule T ′ .inh = F.val, which defines T ′ .inh at the right child of the root from F.va1 at the left child.
- We see edges to node 6 from node 5 for T ′ .inh and from node 4 for F.val, because these values are multiplied to evaluate the attribute inh at node 6.
- Nodes 7 and 8 represent the synthesized attribute syn associated with the occurrences of T ′
- The edge to node 7 from 6 is due to the semantic rule T ′ .syn = T ′ .inh associated with production T ′ → ε.
- The edge to node 8 from 7 is due to a semantic rule associated with production 2.
- Finally, node 9 represents the attribute T.val. The edge from 8 to 9 is due to the semantic rule, T.val = T ′ .syn, associated with production

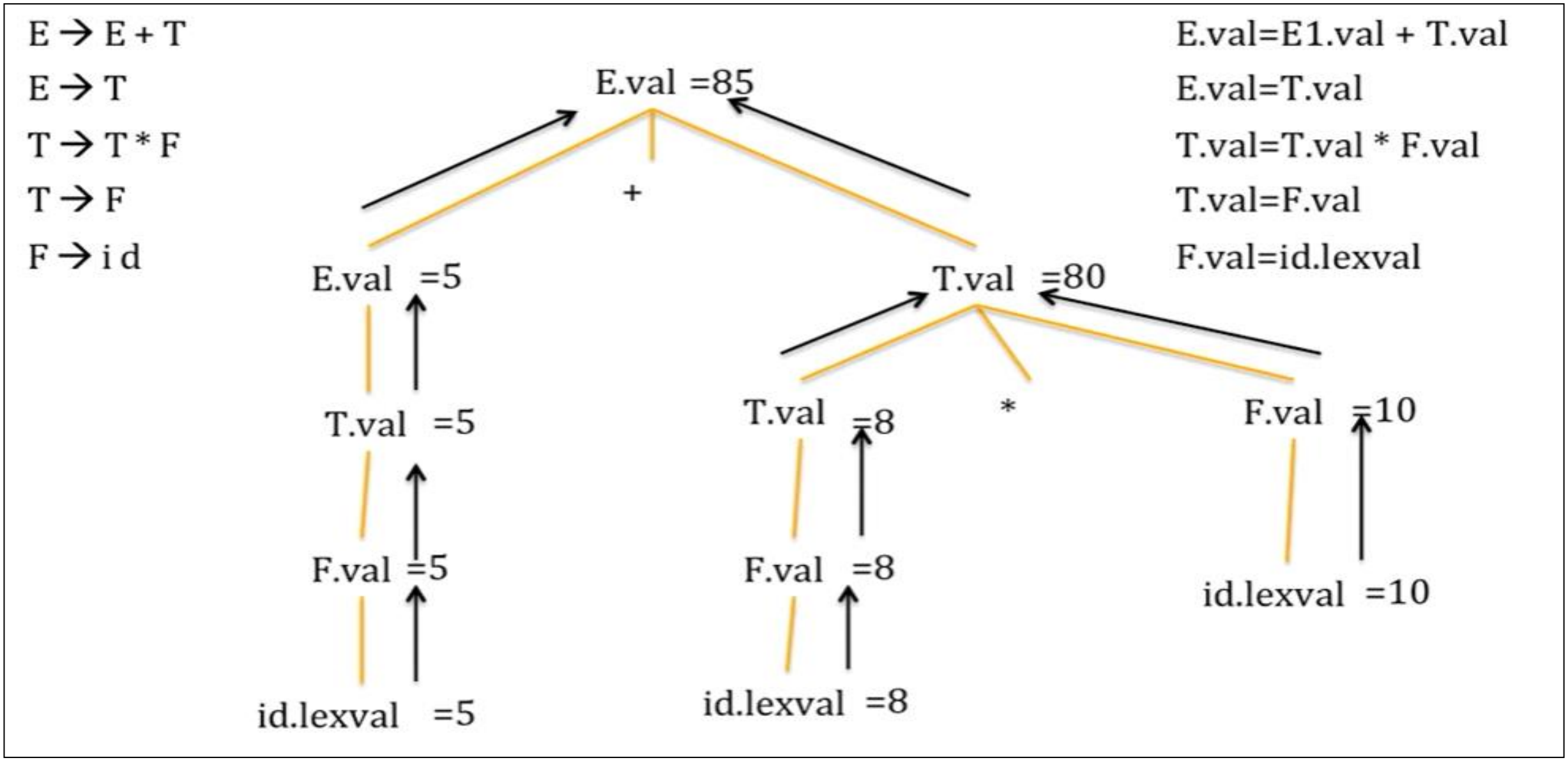
	PRODUCTION	SEMANTIC RULES
1)	$T \rightarrow F\, T'$	$T'.inh = F.val$ $T.val = T'.syn$
2)	$T' \rightarrow * F\, T'_1$	$T'_1.inh = T'.inh \times F.val$ $T'.syn = T'_1.syn$
3)	$T' \rightarrow \epsilon$	$T'.syn = T'.inh$
4)	$F \rightarrow$ **digit**	$F.val =$ **digit**$.lexval$



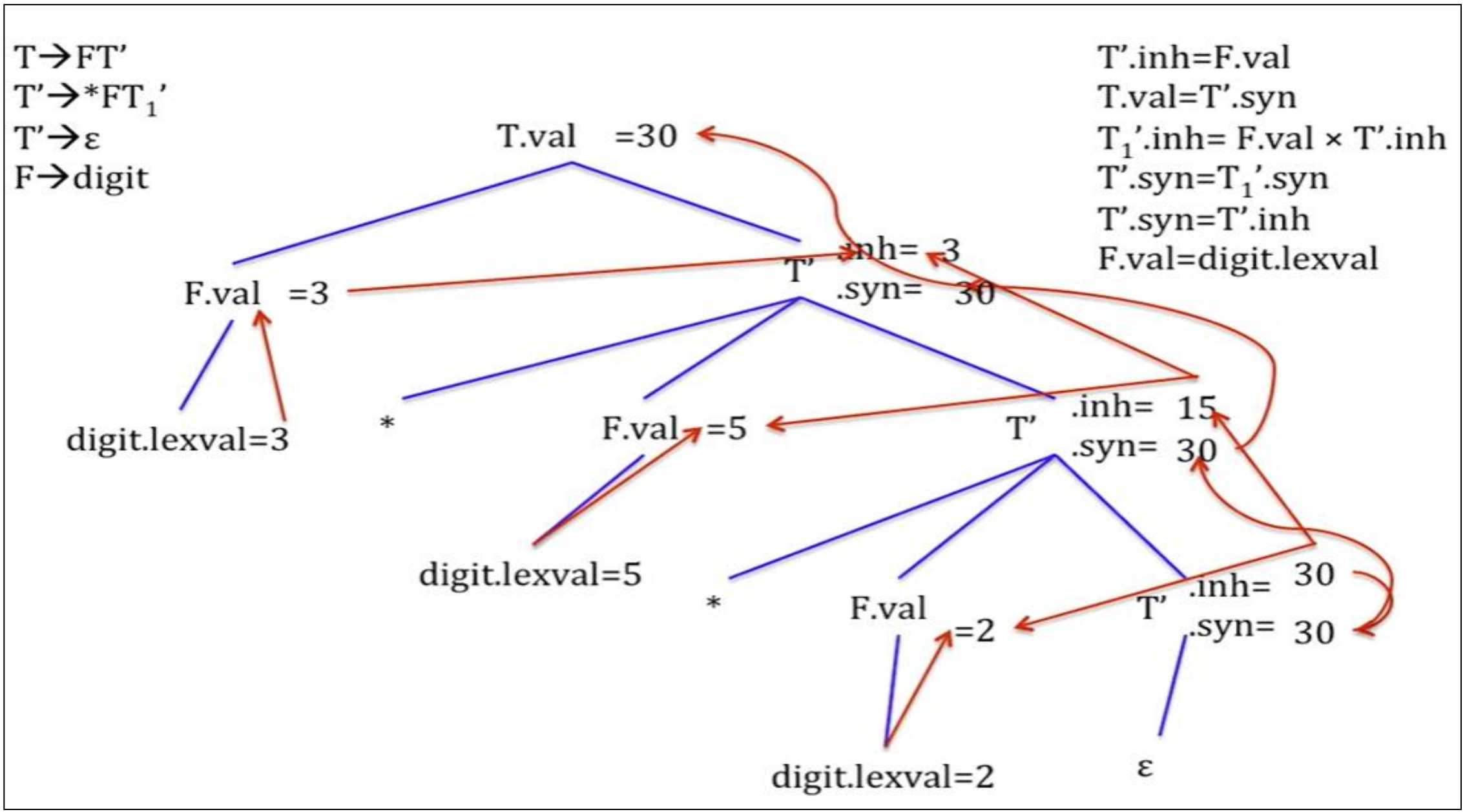


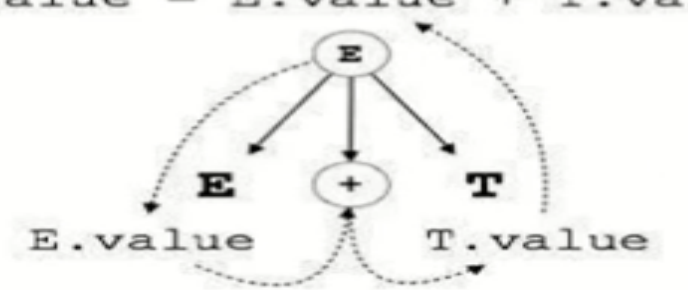
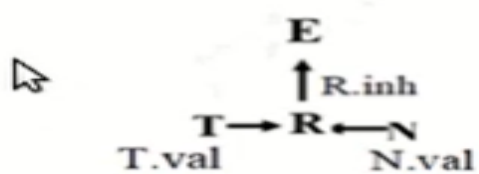
Annotated parse tree for 3 * 5

Expression 5 + 8 *10



Expression 3 * 5 *2



Sr.No.	Synthesized Attributes	Inherited Attributes
1	An attribute is said to be Synthesized attribute if its parse tree node value is determined by the attribute value from its **childrens**.	An attribute is said to be Inherited attribute if its parse tree node value is determined by the attribute value at **parent and/or siblings'** node.
2	The production must have non-terminal as its head.	The production must have non-terminal as a symbol in its body.
3	Synthesized attributes can be contained both the terminals and non-terminals.	Inherited attributes can't be contained by both, it is only contained by non-terminals.
4	It can be evaluated during **bottom-up** traversal of parse tree.	It can be evaluated during **top-down** and sideways traversal of parse tree.
5	Synthesized attributes are used in **S-attributed SDT and L-attributed STD**.	Inherited attributes are used by only **L-attributed SDT.**
6	**e.g.** E → E + T { E.value = E.value + T.value } E → E * T { E.value = E.value * T.value }	**e.g. E → T R N** E.val = R.val R.inh = T.val R.inh = N.val
7	E.value = E.value + T.value 	E.val = R.val R.inh = T.val R.inh = N.val 

- **Attribute grammar**
- This is a special case of context free grammar where additional information is appended to one or more non-terminals in-order to provide context-sensitive information.

- ***Example***
  Given the CFG below;
  $E \rightarrow E + T \{ E.value = E.value + T.value \}$

The right side contains semantic rules that specify how the grammar should be interpreted.
The non-terminal values of E and T are added and their result copied to the non-terminal E.

- Consider the grammar for signed binary numbers

number → signlist
sign → + | −
list → listbit | bit
bit → 0 | 1

- We want to build an attribute grammar that annotates Number with the value it represents.
- First we associate attributes with grammar symbols.

Symbol	Attributes
number	val
sign	neg
list	pos, val
bit	pos, val

Attribute Grammar

Production	Attribute Rule

$number \rightarrow sign\ list$	$list.pos = 0$
	if $sign.neg$:
	$number.val = -list.val$
	else:
	$number.val = -list.val$

$sign \rightarrow +$	$sign.neg =$ false
$sign \rightarrow -$	$sign.neg =$ true
$list \rightarrow bit$	$bit.pos = list.pos$ $list.val = bit.val$
$list_0 \rightarrow list_1 bit$	$list_1.pos = list_0.pos + 1$ $bit.pos = list_0.pos$ $list_0.val = list_1.val + bit.val$
$bit \rightarrow 0$	$bit.val = 0$
$bit \rightarrow 1$	$bit.val = 2^{bit.pos}$